

9주차

서브쿼리와 집합 연산

중첩 질의 · 상관 서브쿼리 · EXISTS · UNION

한국기술교육대학교 경영학부

경영데이터베이스및실습 (CCT770) · 2026학년도 2학기

이번 주를 마치면

목표 ①	서브쿼리를 위치별(스칼라·파생 테이블·중첩)로 구분해 쓴다
목표 ②	단일행·다중행에 맞는 연산자(비교, IN, ANY, ALL)를 고른다
목표 ③	EXISTS로 존재 여부를 판정하고 NOT IN의 NULL 함정을 설명한다
목표 ④	UNION과 UNION ALL의 차이를 알고 두 결과 집합을 결합한다
목표 ⑤	조인과 서브쿼리 중 상황에 맞는 방법을 선택한다

다운유통 실습 DB로 「기준과의 비교」 질문에 답해 봅니다

오늘 다루는 여섯 가지

1 서브쿼리의 개념과 분류

2 WHERE 절 서브쿼리 — 단일행 · IN · ANY/ALL

3 상관 서브쿼리와 EXISTS

4 FROM 절 서브쿼리 (파생 테이블)

5 집합 연산자 — UNION / UNION ALL

6 조인 vs 서브쿼리

서브쿼리 — SQL 문 안의 SELECT

서브쿼리

01

다른 SQL 문 안에 포함된 SELECT
괄호로 감싸서 작성한다

메인 쿼리

02

서브쿼리를 품은 바깥 질의
「주 질의」라고도 한다

언제 쓰나

03

다른 질의의 결과를 기준으로
비교하는 질문에 답할 때

예: 「평균 판매가보다 비싼 상품」, 「주문 이력이 없는 고객」

1. 서브쿼리의 개념과 분류

위치에 따른 분류

위치	명칭	반환 형태	용도
SELECT 절	스칼라 서브쿼리	단일 행 · 단일 열	계산된 컬럼 추가
FROM 절	파생 테이블 (인라인 뷰)	여러 행 · 여러 열	중간 결과를 테이블처럼 사용
WHERE / HAVING 절	중첩 서브쿼리	단일값 / 목록	조건 비교
INSERT/UPDATE 등	DML 서브쿼리	상황에 따라	다른 테이블 값으로 조작

같은 서브쿼리라도 어디에 두느냐에 따라 돌려주는 모양과 쓰임이 달라진다

1. 서브쿼리의 개념과 분류

결과 행 수에 따른 분류

종류	반환	사용 연산자
단일행 서브쿼리	1행	=, <>, >, >=, <, <=
다중행 서브쿼리	여러 행	IN, NOT IN, ANY(SOME), ALL, EXISTS
다중열 서브쿼리	여러 열	(a,b) IN (SELECT x,y ...)

돌아오는 행이 하나인지 여러 개인지가 쓸 수 있는 연산자를 정한다

2. WHERE 절 서브쿼리

단일행 서브쿼리와 스칼라 서브쿼리

```
1 SELECT name, price
2 FROM products
3 WHERE price > (SELECT AVG(price) FROM products);
4
5 SELECT name,
6        price,
7        (SELECT ROUND(AVG(price)) FROM products) AS 전체 평균,
8        price - (SELECT ROUND(AVG(price)) FROM products) AS 차이
9 FROM products
10 WHERE price > (SELECT AVG(price) FROM products);
```

3행

- 1 괄호 안이 먼저 계산
평균 32620 → price > 32620

7행

- 2 스칼라 서브쿼리
모든 행에 같은 값(전체 평균)

8행

- 3 내 가격 - 평균
프리미엄 폭(원)

10행

- 4 WHERE 는 행을 거른다
SELECT 의 서브쿼리는 값을 보여 준다

결과: 무선 전기포트 42000 · 공기청정기 189000 — 기준값을 직접 적지 않으므로 가격이 바뀌어도 질의는 그대로

서브쿼리는 「기준과의 비교」 도구

프리미엄 후보

01

평균보다 비싼 상품은
프리미엄 라인 후보

이탈 방지 대상

02

주문 이력 없는 고객은
이탈 방지 캠페인 대상

하드코딩과 차이

03

WHERE price > 32620 과 달리
평균이 바뀌어도 고칠 필요 없음

기준값(평균)이 바뀌어도 질의를 고칠 필요가 없다 — 하드코딩과의 결정적 차이

2. WHERE 절 서브쿼리

다중행 서브쿼리 — = 대신 IN

```
1 SELECT order_id, cust_id, order_date, status
2 FROM orders
3 WHERE cust_id IN (SELECT cust_id FROM customers WHERE grade = 'VIP');
4
5 -- 단일행 연산자 = 를 쓰면 오류!
6 -- WHERE cust_id = (SELECT cust_id FROM customers WHERE grade = 'VIP')
7 -- ERROR 1242: Subquery returns more than 1 row
```

1

3행

1

IN (목록)

목록 중 하나와 같으면 참

2

6-7행

2

= 는 값 하나만 비교

2행이 돌아오면 ERROR 1242

VIP 2명(C001 한가람 · C004 임수빈) → 주문 6건: 1001·1003·1004·1008·1010·1014 — VIP 가 무엇을 언제 샀는지 추적

2. WHERE 절 서브쿼리

ANY 와 ALL — 목록을 > · < 와 함께

```
1 SELECT name, price FROM products
2 WHERE price > ANY (SELECT price FROM products WHERE category = '식품');
3
4 SELECT name, price FROM products
5 WHERE price > ALL (SELECT price FROM products WHERE category = '식품');
```

1

2행

1

> ANY — 하나라도

최솟값(15,800원)보다 크면 참

2

5행

2

> ALL — 모두

최댓값(24,000원)보다 커야 참

IN 은 = 만 가능 — 다중행 목록을 > · < 와 쓸 때 ANY / ALL · > ALL 결과: 무선 전기포트(42000) · 공기청정기(189000)

ANY · ALL 을 MIN · MAX 로 바꿔 읽기

연산자	의미	동등한 표현
> ANY (목록)	목록의 최솟값보다 크다	> MIN(목록)
< ANY (목록)	목록의 최댓값보다 작다	< MAX(목록)
> ALL (목록)	목록의 최댓값보다 크다	> MAX(목록)
= ANY (목록)	목록 중 하나와 같다	IN (목록)
<> ALL (목록)	목록의 모든 값과 다르다	NOT IN (목록)

> ANY 는 최솟값보다 크면 참, > ALL 은 최댓값보다 커야 참

3. 상관 서브쿼리와 EXISTS

상관 서브쿼리 — 자기 카테고리 평균보다 비싼 상품

```
1 SELECT p.name, p.category, p.price
2 FROM   products p
3 WHERE  p.price > (SELECT AVG(x.price) FROM products x
4                  WHERE x.category = p.category);
```

2행

- 1 바깥 별칭 p
상품 하나하나

3행

- 2 안쪽 별칭 x
같은 products, 역할이 달라 별칭을 달리

4행

- 3 바깥 p 를 참조
바깥 행마다 안쪽이 다시 계산된다

메인 쿼리의 컬럼을 참조하는 서브쿼리 — 이해하기 쉽지만 행마다 반복 실행되므로 성능에 주의

3. 상관 서브쿼리와 EXISTS

상관 서브쿼리의 실행 순서

바깥 행의 수만큼 안쪽 질의가 반복된다



가전은 가전끼리, 식품은 식품끼리 — 데이터가 많으면 느려질 수 있다(GROUP BY 집계 + JOIN 으로도 풀 수 있다)

3. 상관 서브쿼리와 EXISTS

EXISTS / NOT EXISTS — 있는가, 없는가

```
1 SELECT c.cust_id, c.name
2 FROM customers c
3 WHERE EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.cust_id);
4
5 SELECT c.cust_id, c.name
6 FROM customers c
7 WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.cust_id);
```

1

3행

1

주문이 있으면 참

SELECT 1 — 존재만 확인, 값은 안 쓴다

2

7행

2

주문이 하나도 없으면 참

C010 서지안 — 첫 구매 유도 대상

EXISTS 는 첫 주문을 찾는 순간 멈추므로(단축 평가) IN 보다 효율적인 경우가 많다

3. 상관 서브쿼리와 EXISTS

같은 결과를 얻는 세 가지 방법

방법	형태	특징
① NOT EXISTS	WHERE NOT EXISTS (SELECT 1 ...)	NULL 안전, 일반적으로 권장
② 안티 조인	LEFT JOIN ... WHERE o.order_id IS NULL	7주차에서 배운 방법
③ NOT IN	WHERE ... NOT IN (SELECT ...)	서브쿼리 결과에 NULL 이 있으면 항상 공집합

주문 이력이 없는 고객 찾기 — 세 방법 모두 가능하지만 NULL 에 대한 안전성이 다르다

3. 상관 서브쿼리와 EXISTS

NOT IN 함정 — 주문을 담당한 적 없는 직원

```
1 SELECT name FROM employees
2 WHERE emp_id NOT IN (SELECT emp_id FROM orders);
3
4 SELECT e.name
5 FROM employees e
6 WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.emp_id = e.emp_id);
7
8 SELECT name FROM employees
9 WHERE emp_id NOT IN (SELECT emp_id FROM orders WHERE emp_id IS NOT NULL);
```

1

2행

1 결과 0행 (!)

온라인 주문의 emp_id 가 NULL

2

6행

2 대안 ① NOT EXISTS

6명 — 담당 이력은 E002·E003뿐

3

9행

3 대안 ② NULL 제거

목록에서 NULL 을 미리 걸러 낸다

NOT IN 은 「≠ E002 AND ≠ E003 AND ≠ NULL」로 풀리고, ≠ NULL 은 UNKNOWN → 모든 행이 탈락한다

4. FROM 절 서브쿼리

파생 테이블 — 중간 결과를 테이블처럼

```
1 SELECT t.order_id, t.결제금액
2 FROM (
3     SELECT order_id, SUM(qty * unit_price) AS 결제금액
4     FROM   order_items
5     GROUP BY order_id
6 ) AS t
7 WHERE t.결제금액 > (
8     SELECT AVG(총액) FROM (
9         SELECT SUM(qty * unit_price) AS 총액
10        FROM   order_items
11        GROUP BY order_id
12    ) AS s
13 );
```

3-5행

- 1 주문별 결제 금액
품목 금액을 합쳐 주문 총액

6행

- 2 별칭 필수
없으면 오류

8행

- 3 집계 of 집계
SUM 의 AVG 는 한 번 더 감싼다

12행

- 4 여기에도 별칭 s

결과: 주문 1001·1003·1008·1010 (주문 평균 약 68,773원 초과) — 대형 주문 파악

파생 테이블을 쓸 때 기억할 것

별칭은 필수

01

빠뜨리면 「Every derived table must have its own alias」 오류

집계에 다시 조건

02

집계 결과를 한 번 더 걸러야 할 때

집계의 집계

03

주문별 합계의 평균처럼
두 번 집계할 때

경영 의미: 평균보다 큰 주문 = 대형 주문 → 우수 고객 관리 · 재고 우선 배정의 판단 근거

5. 집합 연산자

두 SELECT 결과를 세로로 결합

컬럼 개수

01

두 결과의 컬럼 개수가
같아야 한다

자료형 호환

02

대응하는 컬럼의 자료형이
호환되어야 한다

컬럼 이름

03

첫 번째 SELECT 의
컬럼 이름을 따른다

조인은 옆으로 붙이고, 집합 연산은 위아래로 붙인다

5. 집합 연산자

집합 연산자 네 가지

연산자	의미	중복	MySQL 8.0
UNION	합집합	중복 제거 (정렬 발생)	지원
UNION ALL	합집합	중복 유지 (더 빠름)	지원
INTERSECT	교집합	중복 제거	8.0.31+ 지원
EXCEPT (MINUS)	차집합	중복 제거	8.0.31+ 지원

INTERSECT · EXCEPT 는 MySQL 8.0.31 이상에서 지원된다

5. 집합 연산자

UNION vs UNION ALL — VIP 와 서울 고객

```
1 SELECT cust_id, name FROM customers WHERE grade = 'VIP'
2 UNION
3 SELECT cust_id, name FROM customers WHERE city = '서울';
4
5 SELECT cust_id, name FROM customers WHERE grade = 'VIP'
6 UNION ALL
7 SELECT cust_id, name FROM customers WHERE city = '서울';
```

1

2행

1

UNION — 4행

한사람은 VIP 이면서 서울, 한 번만

2

6행

2

UNION ALL — 5행

한사람이 두 번 나온다

경영 의미: 두 조건 중 하나라도 해당하는 「캠페인 발송 명단」 — 같은 사람에게 두 번 보내지 않는다

5. 집합 연산자

온·오프라인 주문을 채널 표시와 함께

```
1 SELECT order_id, order_date, '온라인' AS 채널
2 FROM orders WHERE emp_id IS NULL
3 UNION ALL
4 SELECT order_id, order_date, '오프라인'
5 FROM orders WHERE emp_id IS NOT NULL
6 ORDER BY order_date;
```

1

1행

- 1 문자열 상수 컬럼
어느 집합에서 왔는지 표시

2

3

3행

- 2 UNION ALL
겹칠 수 없는 두 집합 → 중복 검사 불필요

4

4행

- 3 별칭 생략 가능
컬럼 이름은 첫 SELECT(채널)

6행

- 4 ORDER BY 는 마지막에
결합된 전체를 한 번만 정렬

UNION 은 중복 제거를 위해 정렬하므로, 두 집합이 겹칠 수 없음이 확실하면 UNION ALL 이 훨씬 빠르다

6. 조인 vs 서브쿼리

상황에 맞는 방법 고르기

상황	권장	이유
다른 테이블의 컬럼을 결과에 표시해야 함	JOIN	서브쿼리는 결과 컬럼을 제공하지 못함
존재 여부만 확인 (표시 불필요)	EXISTS	첫 행을 찾으면 즉시 중단하여 효율적
집계 결과와 비교	서브쿼리	WHERE price > (SELECT AVG(price) ...)
1:N 조인으로 행이 중복될 우려	EXISTS / 서브쿼리	조인은 행이 부풀려질 수 있음

결과에 보여야 하면 JOIN, 있는지만 알면 EXISTS, 집계와 비교하면 서브쿼리

자주 틀리는 지점

■ 증상

! **ERROR 1242: Subquery returns more than 1 row**

! **NOT IN 결과가 0행**

! **Every derived table must have its own alias**

! **합친 매출이 줄어 있다**

■ 원인

→ 여러 행이 돌아오는데 = 로 비교

→ 서브쿼리 목록에 NULL 이 섞임

→ FROM 절 서브쿼리에 별칭이 없음

→ UNION 이 같은 금액의 주문을 지움

■ 해결

✓ **IN (또는 ANY · ALL) 을 쓴다**

✓ **NOT EXISTS 또는 WHERE 컬럼 IS NOT NULL**

✓ **닫는 괄호 뒤에 AS t 를 붙인다**

✓ **겹칠 수 없는 집합은 UNION ALL**

확인 퀴즈

서브쿼리 결과에 NULL이 포함되어 있을 때 결과가 항상 공집합이 될 수 있는 연산자는?

① EXISTS

② NOT EXISTS

③ NOT IN

④ IN

정답 및 해설

정답

③ NOT IN

해설

NOT IN은 NULL과의 비교가 UNKNOWN이 되어 전체 결과가 비어 버립니다. NOT EXISTS를 사용하는 것이 안전합니다. orders.emp_id처럼 NULL이 섞인 컬럼을 서브쿼리로 쓸 때 주의해야 합니다.

핵심 요약

핵심 ①	위치에 따라 스칼라(SELECT) · 파생 테이블(FROM) · 중첩(WHERE)
핵심 ②	단일행에는 비교 연산자, 다중행에는 IN · ANY · ALL · EXISTS
핵심 ③	> ANY 는 최솟값보다 크면 참, > ALL 은 최댓값보다 커야 참
핵심 ④	NOT IN 은 목록에 NULL 이 있으면 공집합 — NOT EXISTS 를 쓴다
핵심 ⑤	UNION 은 중복 제거, UNION ALL 은 중복 유지하며 더 빠르다

orders.emp_id 처럼 NULL 이 있는 컬럼을 목록으로 쓸 때는 NOT EXISTS

다음 주차 예고

10주차 · 뷰·인덱스·권한(DCL)

01

뷰

자주 보는 리포트를
이름 하나로

02

인덱스

조회는 빠르게
쓰기는 느리게

03

EXPLAIN

인덱스를 타는지
실행 계획 확인

04

권한(DCL)

GRANT · REVOKE · ROLE
필요한 최소한만

9주차 마무리

학습 마무리

오늘 배운 질의를 다운유통 실습 DB에서 직접 실행해 봅니다

✓ 단일행 · 다중행 서브쿼리와 연산자

✓ 상관 서브쿼리와 EXISTS

✓ NOT IN 의 NULL 함정

✓ UNION 과 UNION ALL

이어서는 활동

9주차 실습 문제 · 퀴즈 7